



COS20028 – BIG DATA ARCHITECTURE AND APPLICATION

Dr Pei-Wei Tsai (Lecturer, Tutor, and Unit Convenor)
ptsai@swin.edu.au, EN508d



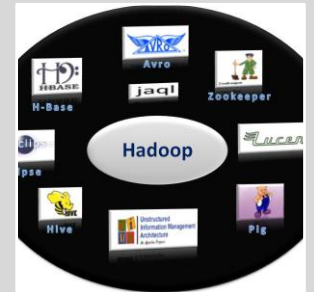


Week 2



MapReduce

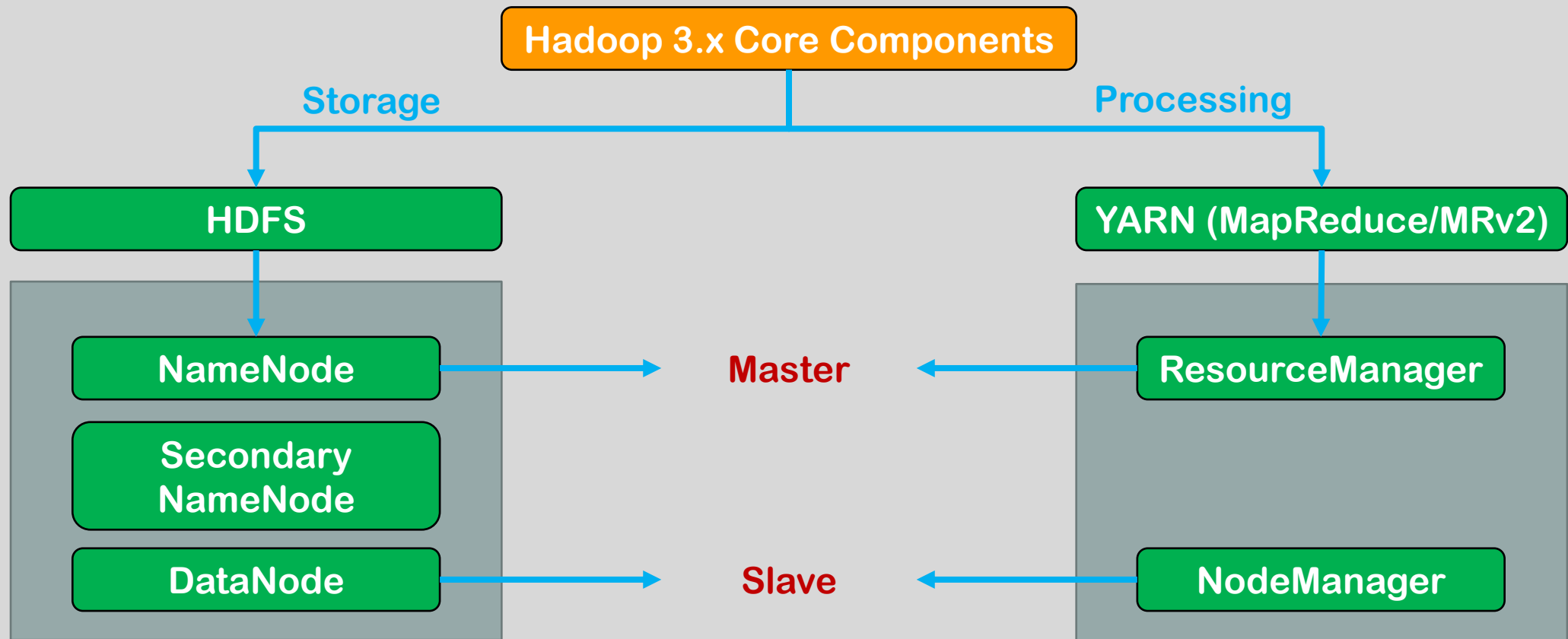
Hadoop
Ecosystem





HADOOP MAPREDUCE

Hadoop 3.x Daemons





What does Hadoop
MapReduce do?

- Hadoop MapReduce is the **processing component** of Apache Hadoop.
- It processes data parallelly in distributed environment.

**MapReduce
is used in**

Index and Search

Classification

Recommendation

Analytics

MapReduce Features



A Program Model

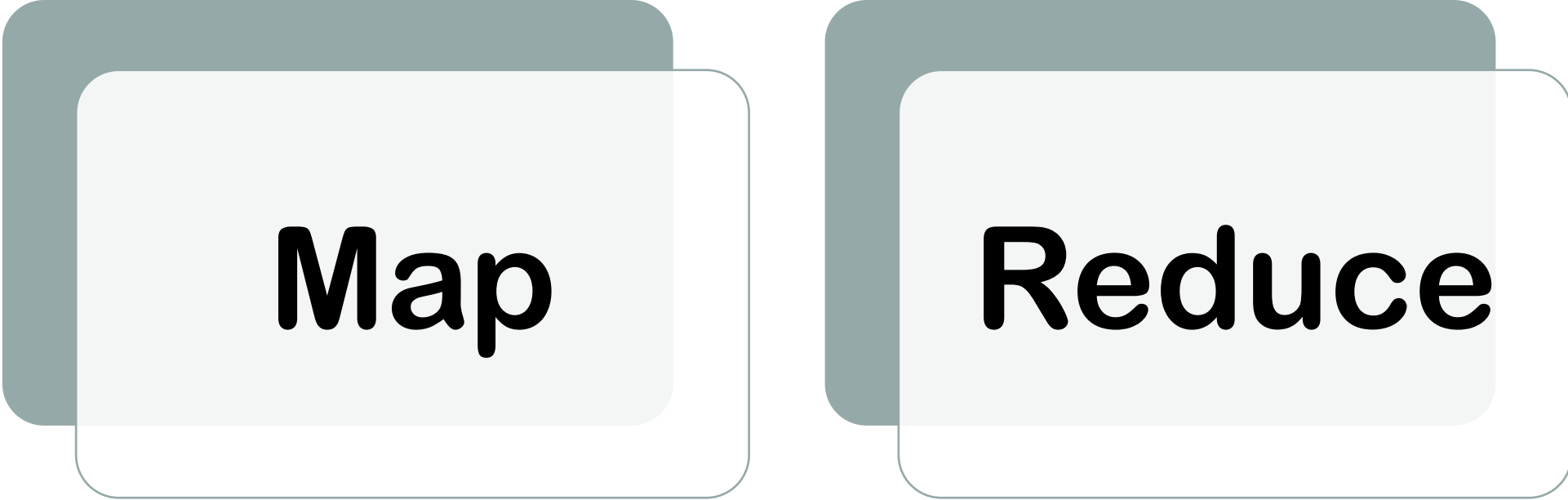


**Large Scale Distributed
Model**



Parallel Programming

MapReduce Functions



The diagram illustrates the MapReduce process. At the top, a dark gray rectangular box contains the title "MapReduce Functions" in white text. Below this, there are two identical visual elements side-by-side. Each element consists of a light gray rounded rectangle with a thin gray border, which is slightly offset to the right and bottom from a darker gray rounded rectangle behind it. The word "Map" is centered in the light gray rectangle on the left, and the word "Reduce" is centered in the light gray rectangle on the right.

Map

Reduce

MapReduce is Implemented in

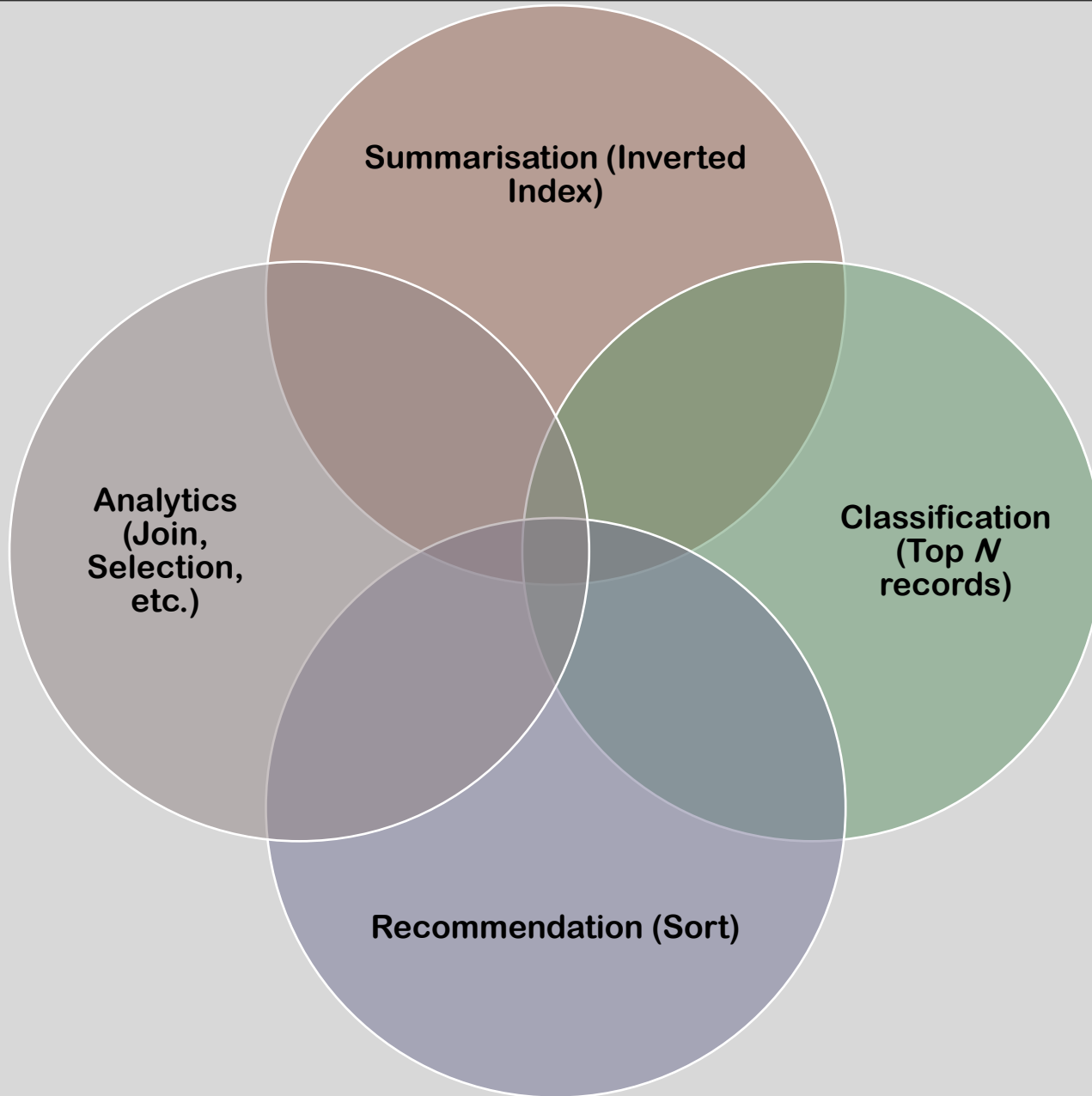
The Google logo is presented within a graphic consisting of two overlapping rounded rectangles. The front rectangle is light gray with a thin dark gray border, and the word "Google" is centered in it. The back rectangle is a slightly darker shade of gray and is offset to the top-left.

Google

The Apache Hadoop logo is presented within a graphic consisting of two overlapping rounded rectangles. The front rectangle is light gray with a thin dark gray border, and the words "Apache" and "Hadoop" are centered in it. The back rectangle is a slightly darker shade of gray and is offset to the top-left.

**Apache
Hadoop**

MapReduce Design Pattern



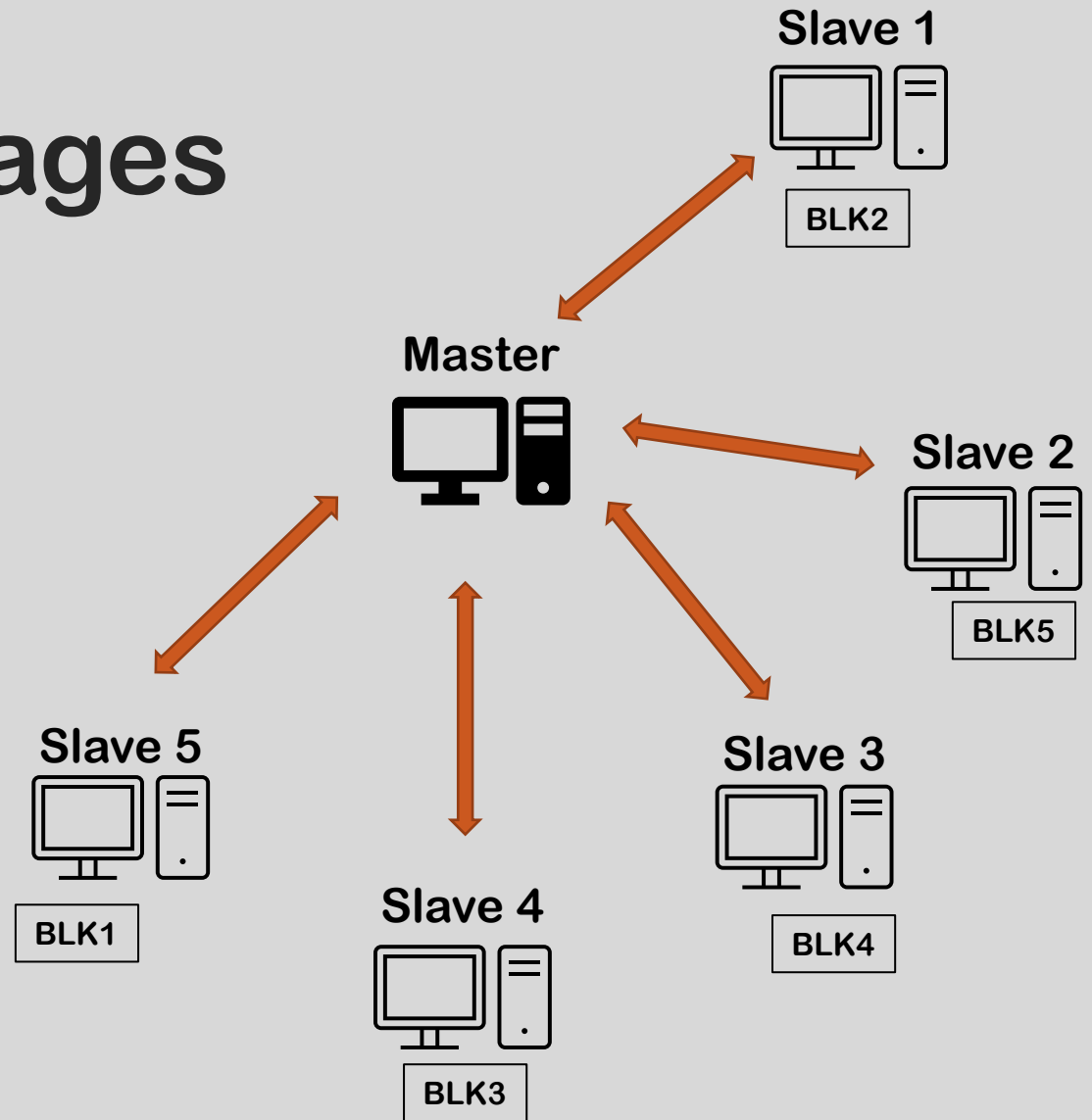
MapReduce Advantages

1. Parallel Processing

- Data can be processed in parallel.

2. Data Locality (Processing to Storage)

- Moving data to processing is very costly.
- In MapReduce, it is the “process” to be moved to the data.





WITH AND WITHOUT MAPREDUCE

Case Study: Financial Report

- A chain store X has four retailer shops in Victoria.
- If the headquarter (HQ) wants to know the latest financial report of the company, which of the following option is the most efficient way to do?

(A)

Ask all retailer shops to send their selling and stacking raw data back to HQ for finding the result.

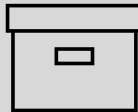
(B)

Ask all retailer shops to prepare the summary report based on their selling and stacking raw data and send the report back to the HQ.

Case Study: Financial Report

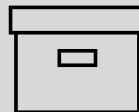
(A) Ask all retailer shops to send their selling and stacking raw data back to HQ for finding the result.

Retailer 1

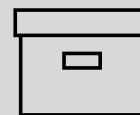


HQ

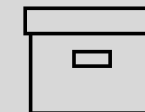
Retailer 2



Retailer 3



Retailer 4



Case Study: Financial Report

(B) Ask all retailer shops to prepare the summary report based on their selling and stacking raw data and send the report back to the HQ.

Retailer 1



HQ



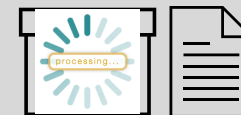
Retailer 2



Retailer 3



Retailer 4

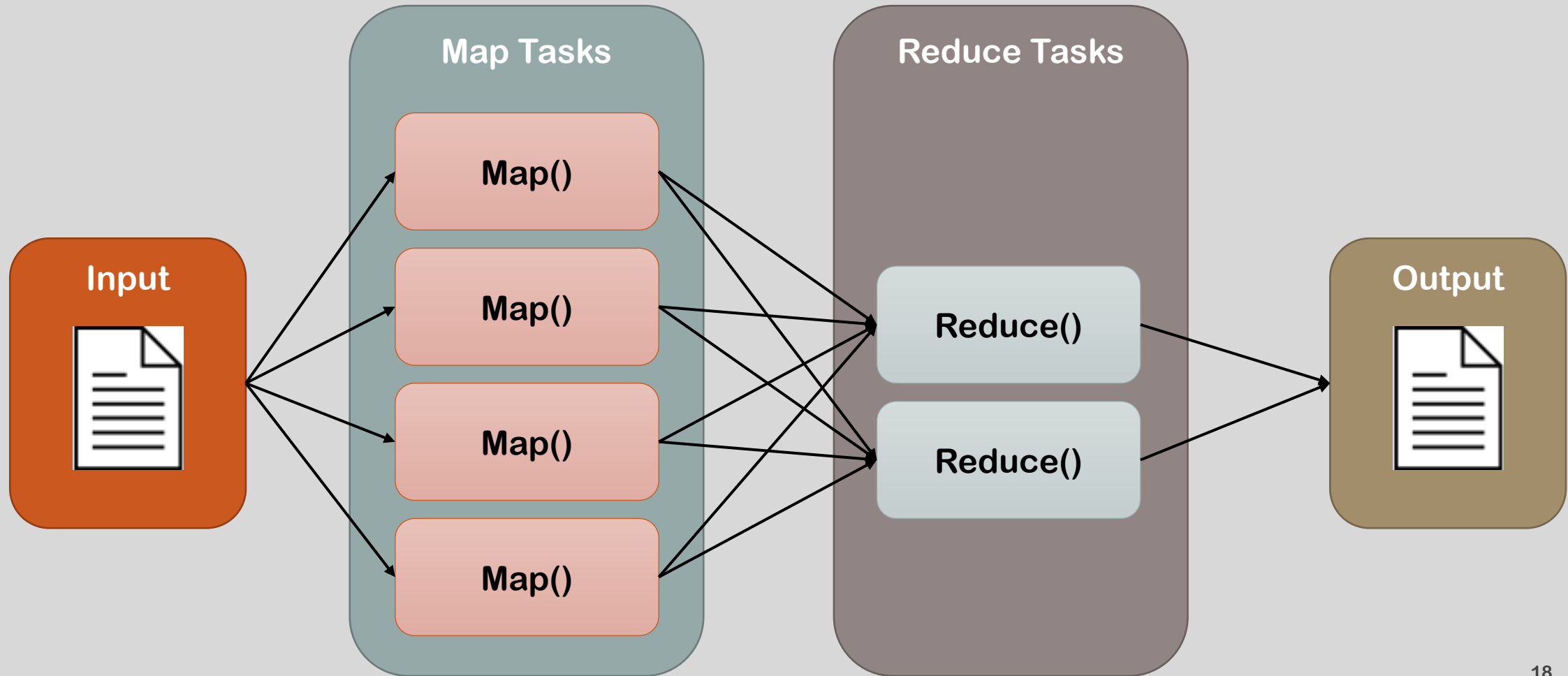




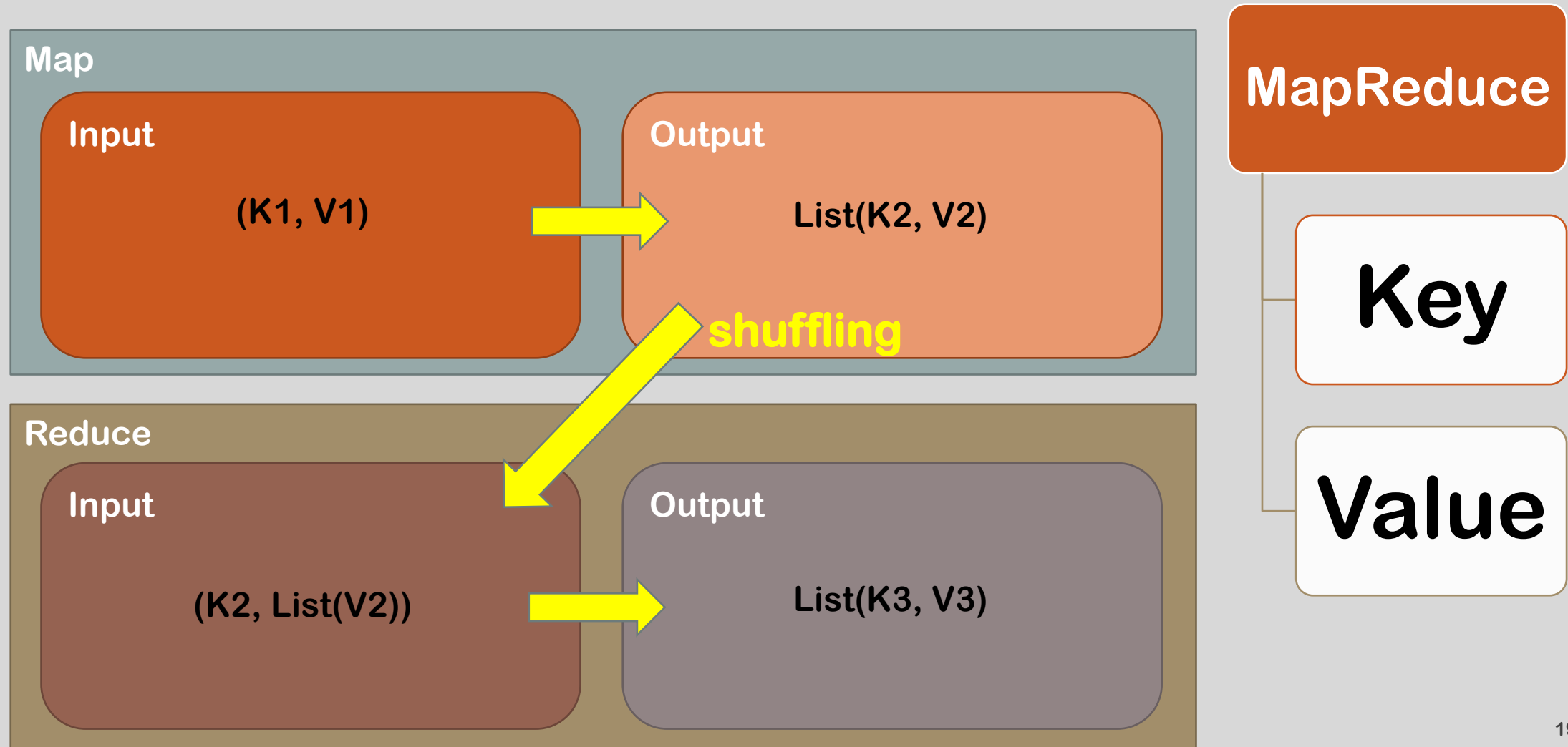
BIG DATA IS
WATCHING YOU

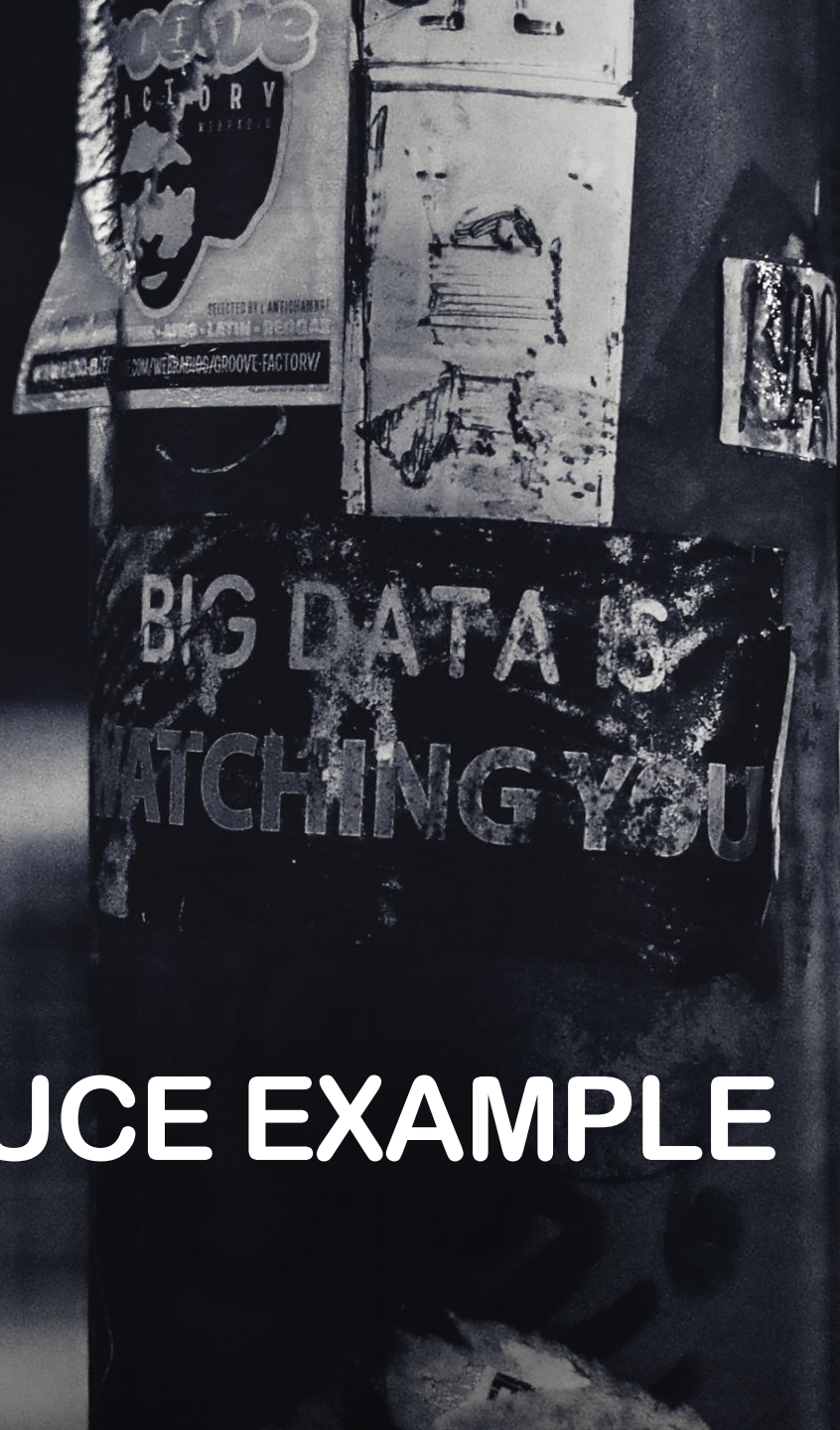
HOW MAPREDUCE WORKS?

MapReduce Framework



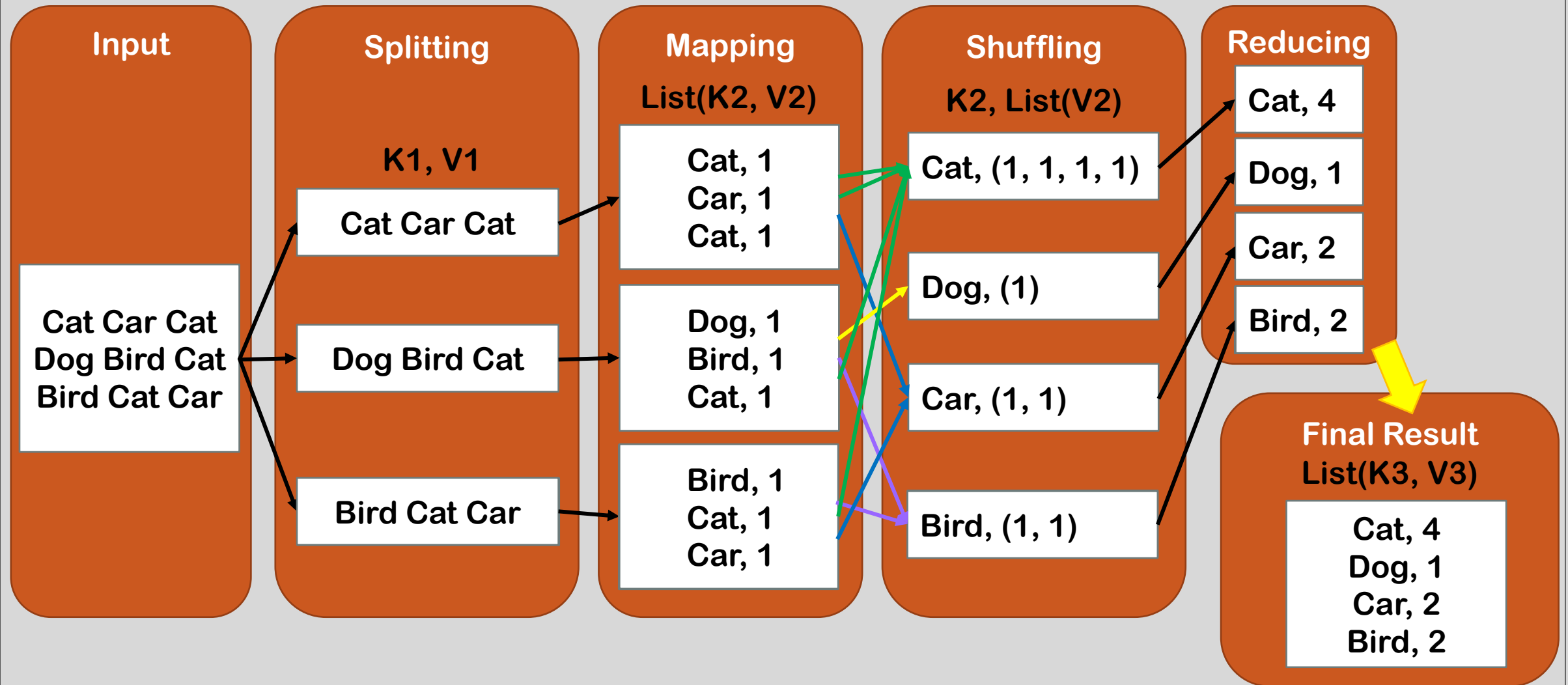
MapReduce Program Anatomy





MAPREDUCE EXAMPLE

Word Count Process





MAPREDUCE EXAMPLE 2

MapReduce Example 2

- Depending on your definition of the map function, the output can be very different.
- Instead of using the word as the key, let's see how it goes if we use the length of the word as the key.

MapReduce Example 2

- Original text:
 - I have a cat. She is ten years old, now! When I was a student, I took her in from the street. She was a stray cat.

1: I
4: have
1: a
3: cat
1: .
3: She
2: is
3: ten
5: years
3: old
1: ,
3: now
1: !

4: when
1: I
3: was
1: a
7: student
1: ,
1: I
4: took
3: her
2: in
4: from
3: the
6: street
1: .

3: she
3: was
1: a
5: stray
3: cat
1: .

MapReduce Example 2

- After grouping:

1: [I a . , !]
2: [is in]
3: [cat she ten old now was her the]
4: [have when took from]
5: [years stray]
6: [street]
7: [student]

1: I
4: have
1: a
3: cat
1: .
3: She
2: is
3: ten
5: years
3: old
1: ,
3: now
1: !

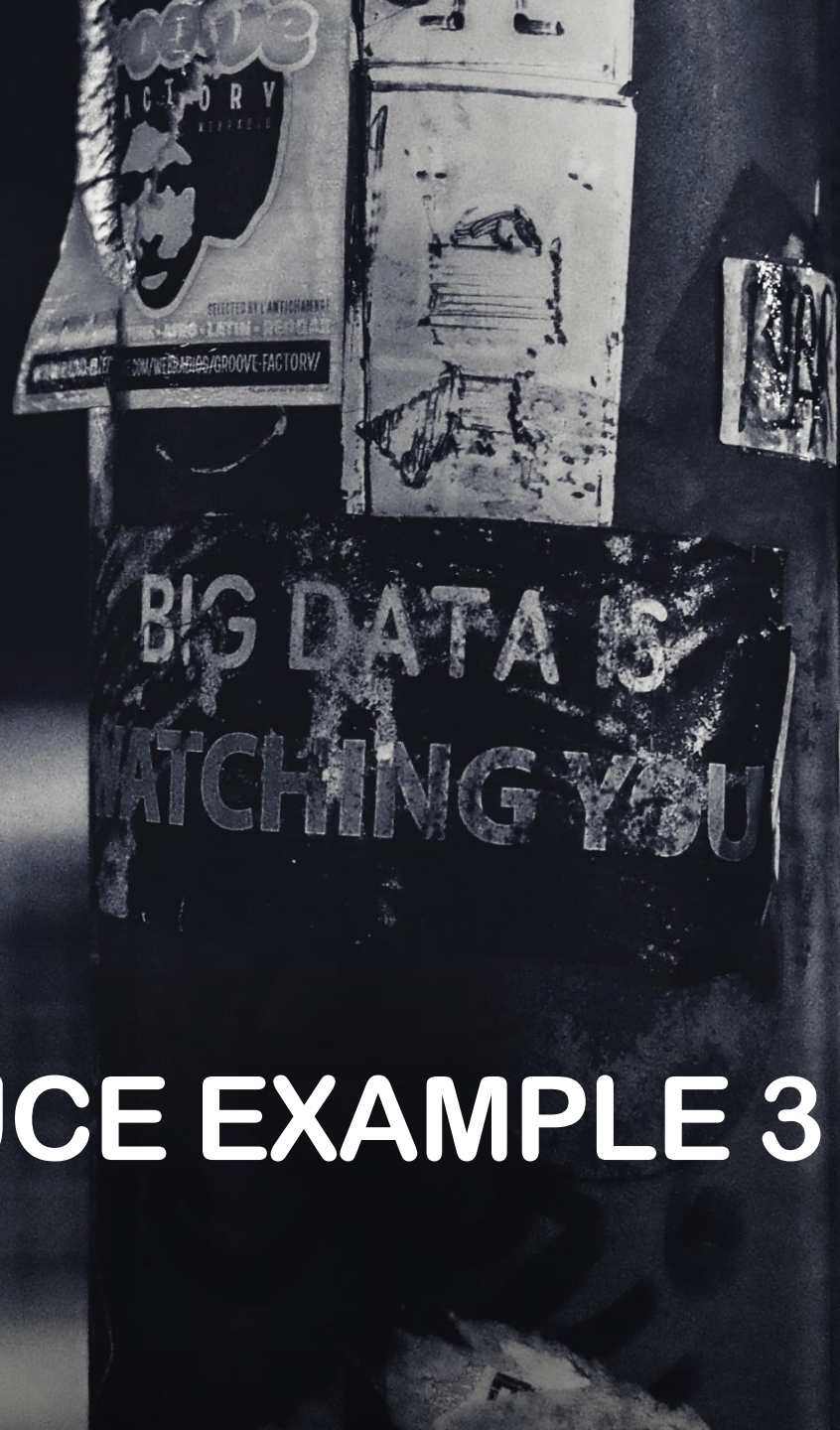
4: when
1: I
3: was
1: a
7: student
1: ,
1: I
4: took
3: her
2: in
4: from
3: the
6: street
1: .

3: she
3: was
1: a
5: stray
3: cat
1: .

MapReduce Example 2

- Each of these lines would be passed as an argument to the reduce function.

1: [I a . , !]	1: 5
2: [is in]	2: 2
3: [cat she ten old now was her the]	3: 8
4: [have when took from]	4: 4
5: [years stray]	5: 2
6: [street]	6: 1
7: [student]	7: 1



MAPREDUCE EXAMPLE 3

Social Media Application

- Social media platforms such as Facebook, LinkedIn, etc. have lists of friends (bi-directional graph).
- These large platforms prefer to pre-do the calculation while they can to reduce the load of the server.
- A common feature
 - You and ooo have xxx friends in common.

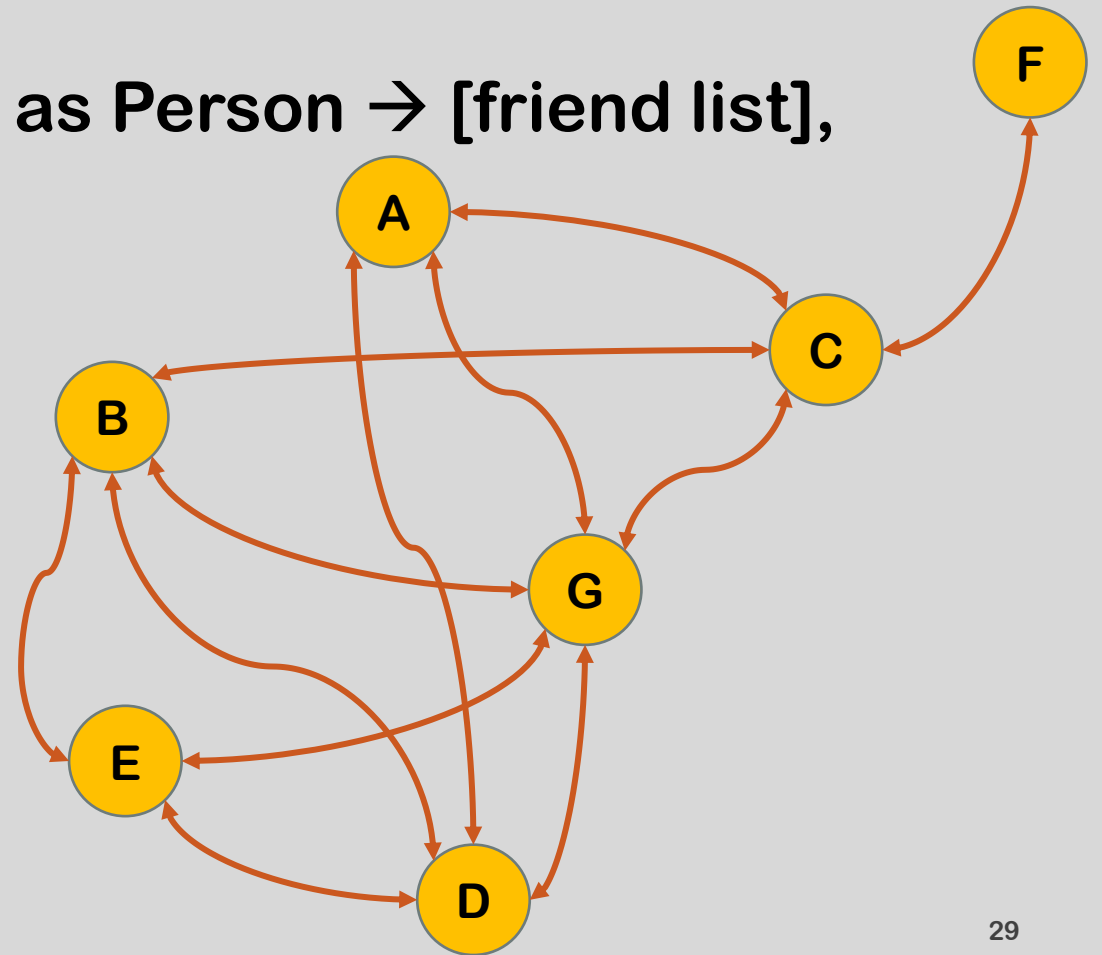
Friends
396 (22 mutual)

Social Media Application

- Assume the friends are stored as $\text{Person} \rightarrow [\text{friend list}]$, a possible example can be:

$A \rightarrow [C\ D\ G]$
 $B \rightarrow [C\ D\ E\ G]$
 $C \rightarrow [A\ B\ F\ G]$
 $D \rightarrow [A\ B\ E\ G]$
 $E \rightarrow [B\ D\ G]$
 $F \rightarrow [C]$
 $G \rightarrow [A\ B\ C\ D\ E]$

Note that X represents the key and Y represents the values.

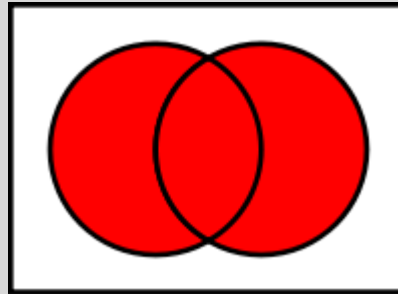


Background Knowledge: Set Operation

- In set theory, a set is defined as a collection of some objects. For example: $M = \{a, b, c, f\}$ and $N = \{b, e\}$.

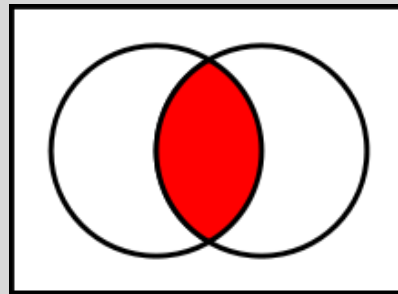
- **Union:**

- $M \cup N = \{a, b, c, e, f\}$



- **Intersection:**

- $M \cap N = \{b\}$



Social Media Application

$(A B) \rightarrow (A \cap B) \cup B \rightarrow [C D G B] \rightarrow [B C D G]$
 $(A C) \rightarrow (A \cap C) \cup C \rightarrow [G C] \rightarrow [C G]$
 $(A D) \rightarrow (A \cap D) \cup D \rightarrow [G D] \rightarrow [D G]$
 $(A E) \rightarrow (A \cap E) \cup E \rightarrow [D G E] \rightarrow [D E G]$
 $(A F) \rightarrow (A \cap F) \cup F \rightarrow [C F] \rightarrow [C F]$
 $(A G) \rightarrow (A \cap G) \cup G \rightarrow [C D G] \rightarrow [C D G]$

$A \rightarrow [C D G]$
 $B \rightarrow [C D E G]$
 $C \rightarrow [A B F G]$
 $D \rightarrow [A B E G]$
 $E \rightarrow [B D G]$
 $F \rightarrow [C]$
 $G \rightarrow [A B C D E]$

$(A B) \rightarrow (A \cap B) \cup B$
 $\rightarrow [C D G] \cup B$
 $\rightarrow [C D G B]$
 $\rightarrow [B C D G]$

Social Media Application

$(AB) \rightarrow (A \cap B) \cup B \rightarrow [CDGB] \rightarrow [BCDG]$
 $(AC) \rightarrow (A \cap C) \cup C \rightarrow [GC] \rightarrow [CG]$
 $(AD) \rightarrow (A \cap D) \cup D \rightarrow [GD] \rightarrow [DG]$
 $(AE) \rightarrow (A \cap E) \cup E \rightarrow [DGE] \rightarrow [DEG]$
 $(AF) \rightarrow (A \cap F) \cup F \rightarrow [CF] \rightarrow [CF]$
 $(AG) \rightarrow (A \cap G) \cup G \rightarrow [CDG] \rightarrow [CDG]$

$(DA) \rightarrow (D \cap A) \cup A \rightarrow [GA] \rightarrow [AG]$
 $(DB) \rightarrow (D \cap B) \cup B \rightarrow [EGB] \rightarrow [BEG]$
 $(DC) \rightarrow (D \cap C) \cup C \rightarrow [ABGC] \rightarrow [ABCG]$
 $(DE) \rightarrow (D \cap E) \cup E \rightarrow [BGE] \rightarrow [BEG]$
 $(DF) \rightarrow (D \cap F) \cup F \rightarrow [F] \rightarrow [F]$
 $(DG) \rightarrow (D \cap G) \cup G \rightarrow [ABEG] \rightarrow [ABEG]$

$A \rightarrow [CDG]$
 $B \rightarrow [CDEG]$
 $C \rightarrow [ABFG]$
 $D \rightarrow [ABEG]$
 $E \rightarrow [BDG]$
 $F \rightarrow [C]$
 $G \rightarrow [ABCDE]$

$(BA) \rightarrow (B \cap A) \cup A \rightarrow [CDGA] \rightarrow [ACDG]$
 $(BC) \rightarrow (B \cap C) \cup C \rightarrow [GC] \rightarrow [CG]$
 $(BD) \rightarrow (B \cap D) \cup D \rightarrow [EGD] \rightarrow [DEG]$
 $(BE) \rightarrow (B \cap E) \cup E \rightarrow [DGE] \rightarrow [DEG]$
 $(BF) \rightarrow (B \cap F) \cup F \rightarrow [CF] \rightarrow [CF]$
 $(BG) \rightarrow (B \cap G) \cup G \rightarrow [CDEG] \rightarrow [CDEG]$

$(EA) \rightarrow (E \cap A) \cup A \rightarrow [DGA] \rightarrow [ADG]$
 $(EB) \rightarrow (E \cap B) \cup B \rightarrow [DGB] \rightarrow [BDG]$
 $(EC) \rightarrow (E \cap C) \cup C \rightarrow [BGC] \rightarrow [BCG]$
 $(ED) \rightarrow (E \cap D) \cup D \rightarrow [BGD] \rightarrow [BDG]$
 $(EF) \rightarrow (E \cap F) \cup F \rightarrow [F] \rightarrow [F]$
 $(EG) \rightarrow (E \cap G) \cup G \rightarrow [BDG] \rightarrow [BDG]$

$(GA) \rightarrow (G \cap A) \cup A \rightarrow [CDA] \rightarrow [ACD]$
 $(GB) \rightarrow (G \cap B) \cup B \rightarrow [CDEB] \rightarrow [BCDE]$
 $(GC) \rightarrow (G \cap C) \cup C \rightarrow [ABC] \rightarrow [ABC]$
 $(GD) \rightarrow (G \cap D) \cup D \rightarrow [ABED] \rightarrow [ABDE]$
 $(GE) \rightarrow (G \cap E) \cup E \rightarrow [BDE] \rightarrow [BDE]$
 $(GF) \rightarrow (G \cap F) \cup F \rightarrow [CF] \rightarrow [CF]$

$(CA) \rightarrow (C \cap A) \cup A \rightarrow [GA] \rightarrow [AG]$
 $(CB) \rightarrow (C \cap B) \cup B \rightarrow [GB] \rightarrow [BG]$
 $(CD) \rightarrow (C \cap D) \cup D \rightarrow [ABGD] \rightarrow [ABDG]$
 $(CE) \rightarrow (C \cap E) \cup E \rightarrow [BGE] \rightarrow [BEG]$
 $(CF) \rightarrow (C \cap F) \cup F \rightarrow [F] \rightarrow [F]$
 $(CG) \rightarrow (C \cap G) \cup G \rightarrow [ABG] \rightarrow [ABG]$

$(FA) \rightarrow (F \cap A) \cup A \rightarrow [CA] \rightarrow [AC]$
 $(FB) \rightarrow (F \cap B) \cup B \rightarrow [CB] \rightarrow [BC]$
 $(FC) \rightarrow (F \cap C) \cup C \rightarrow [C] \rightarrow [C]$
 $(FD) \rightarrow (F \cap D) \cup D \rightarrow [D] \rightarrow [D]$
 $(FE) \rightarrow (F \cap E) \cup E \rightarrow [E] \rightarrow [E]$
 $(FG) \rightarrow (F \cap G) \cup G \rightarrow [CG] \rightarrow [CG]$

Social Media Application

$(AB) \rightarrow (A \cap B) \cup B \rightarrow [CDGB] \rightarrow [BCDG]$
 $(AC) \rightarrow (A \cap C) \cup C \rightarrow [GC] \rightarrow [CG]$
 $(AD) \rightarrow (A \cap D) \cup D \rightarrow [GD] \rightarrow [DG]$
 $(AE) \rightarrow (A \cap E) \cup E \rightarrow [DGE] \rightarrow [DEG]$
 $(AF) \rightarrow (A \cap F) \cup F \rightarrow [CF] \rightarrow [CF]$
 $(AG) \rightarrow (A \cap G) \cup G \rightarrow [CDG] \rightarrow [CDG]$

$(AB) \rightarrow (B \cap A) \cup A \rightarrow [CDGA] \rightarrow [ACDG]$
 $(BC) \rightarrow (B \cap C) \cup C \rightarrow [GC] \rightarrow [CG]$
 $(BD) \rightarrow (B \cap D) \cup D \rightarrow [EGD] \rightarrow [DEG]$
 $(BE) \rightarrow (B \cap E) \cup E \rightarrow [DGE] \rightarrow [DEG]$
 $(BF) \rightarrow (B \cap F) \cup F \rightarrow [CF] \rightarrow [CF]$
 $(BG) \rightarrow (B \cap G) \cup G \rightarrow [CDEG] \rightarrow [CDEG]$

$(AC) \rightarrow (C \cap A) \cup A \rightarrow [GA] \rightarrow [AG]$
 $(BC) \rightarrow (C \cap B) \cup B \rightarrow [GB] \rightarrow [BG]$
 $(CD) \rightarrow (C \cap D) \cup D \rightarrow [ABGD] \rightarrow [ABDG]$
 $(CE) \rightarrow (C \cap E) \cup E \rightarrow [BGE] \rightarrow [BEG]$
 $(CF) \rightarrow (C \cap F) \cup F \rightarrow [F] \rightarrow [F]$
 $(CG) \rightarrow (C \cap G) \cup G \rightarrow [ABG] \rightarrow [ABG]$

$(AB) \rightarrow [BCDG], [ACDG]$

$(AD) \rightarrow (D \cap A) \cup A \rightarrow [GA] \rightarrow [AG]$
 $(BD) \rightarrow (D \cap B) \cup B \rightarrow [EGB] \rightarrow [BEG]$
 $(CD) \rightarrow (D \cap C) \cup C \rightarrow [ABGC] \rightarrow [ABCG]$
 $(DE) \rightarrow (D \cap E) \cup E \rightarrow [BGE] \rightarrow [BEG]$
 $(DF) \rightarrow (D \cap F) \cup F \rightarrow [F] \rightarrow [F]$
 $(DG) \rightarrow (D \cap G) \cup G \rightarrow [ABEG] \rightarrow [ABEG]$

$(AE) \rightarrow (E \cap A) \cup A \rightarrow [DGA] \rightarrow [ADG]$
 $(BE) \rightarrow (E \cap B) \cup B \rightarrow [DGB] \rightarrow [BDG]$
 $(CE) \rightarrow (E \cap C) \cup C \rightarrow [BGC] \rightarrow [BCG]$
 $(DE) \rightarrow (E \cap D) \cup D \rightarrow [BGD] \rightarrow [BDG]$
 $(EF) \rightarrow (E \cap F) \cup F \rightarrow [F] \rightarrow [F]$
 $(EG) \rightarrow (E \cap G) \cup G \rightarrow [BDG] \rightarrow [BDG]$

$(AF) \rightarrow (F \cap A) \cup A \rightarrow [CA] \rightarrow [AC]$
 $(BF) \rightarrow (F \cap B) \cup B \rightarrow [CB] \rightarrow [BC]$
 $(CF) \rightarrow (F \cap C) \cup C \rightarrow [C] \rightarrow [C]$
 $(DF) \rightarrow (F \cap D) \cup D \rightarrow [D] \rightarrow [D]$
 $(EF) \rightarrow (F \cap E) \cup E \rightarrow [E] \rightarrow [E]$
 $(FG) \rightarrow (F \cap G) \cup G \rightarrow [CG] \rightarrow [CG]$

$A \rightarrow [CDG]$
 $B \rightarrow [CDEG]$
 $C \rightarrow [ABFG]$
 $D \rightarrow [ABEG]$
 $E \rightarrow [BDG]$
 $F \rightarrow [C]$
 $G \rightarrow [ABCDE]$

$(AG) \rightarrow (G \cap A) \cup A \rightarrow [CDA] \rightarrow [ACD]$
 $(BG) \rightarrow (G \cap B) \cup B \rightarrow [CDEB] \rightarrow [BCDE]$
 $(CG) \rightarrow (G \cap C) \cup C \rightarrow [ABC] \rightarrow [ABC]$
 $(DG) \rightarrow (G \cap D) \cup D \rightarrow [ABED] \rightarrow [ABDE]$
 $(EG) \rightarrow (G \cap E) \cup E \rightarrow [BDE] \rightarrow [BDE]$
 $(FG) \rightarrow (G \cap F) \cup F \rightarrow [CF] \rightarrow [CF]$

Social Media Application

After Reduce

$A \rightarrow [C D G]$

$B \rightarrow [C D E G]$

$C \rightarrow [A B F G]$

$D \rightarrow [A B E G]$

$E \rightarrow [B D G]$

$F \rightarrow [C]$

$G \rightarrow [A B C D E]$

$(A B) \rightarrow ([B C D G] \cap [A C D G]) \rightarrow [C D G]$

$(A C) \rightarrow ([C G] \cap [A G]) \rightarrow [G]$

$(A D) \rightarrow [D G]$

$(A E) \rightarrow [D G]$

$(A F) \rightarrow [C]$

$(A G) \rightarrow [C D]$

$(B C) \rightarrow [G]$

$(B D) \rightarrow [E G]$

$(B E) \rightarrow [D G]$

$(B F) \rightarrow [C]$

$(B G) \rightarrow [C D E]$

$(C D) \rightarrow [A B G]$

$(C E) \rightarrow [B G]$

$(C F) \rightarrow []$

$(C G) \rightarrow [A B]$

$(D E) \rightarrow [B G]$

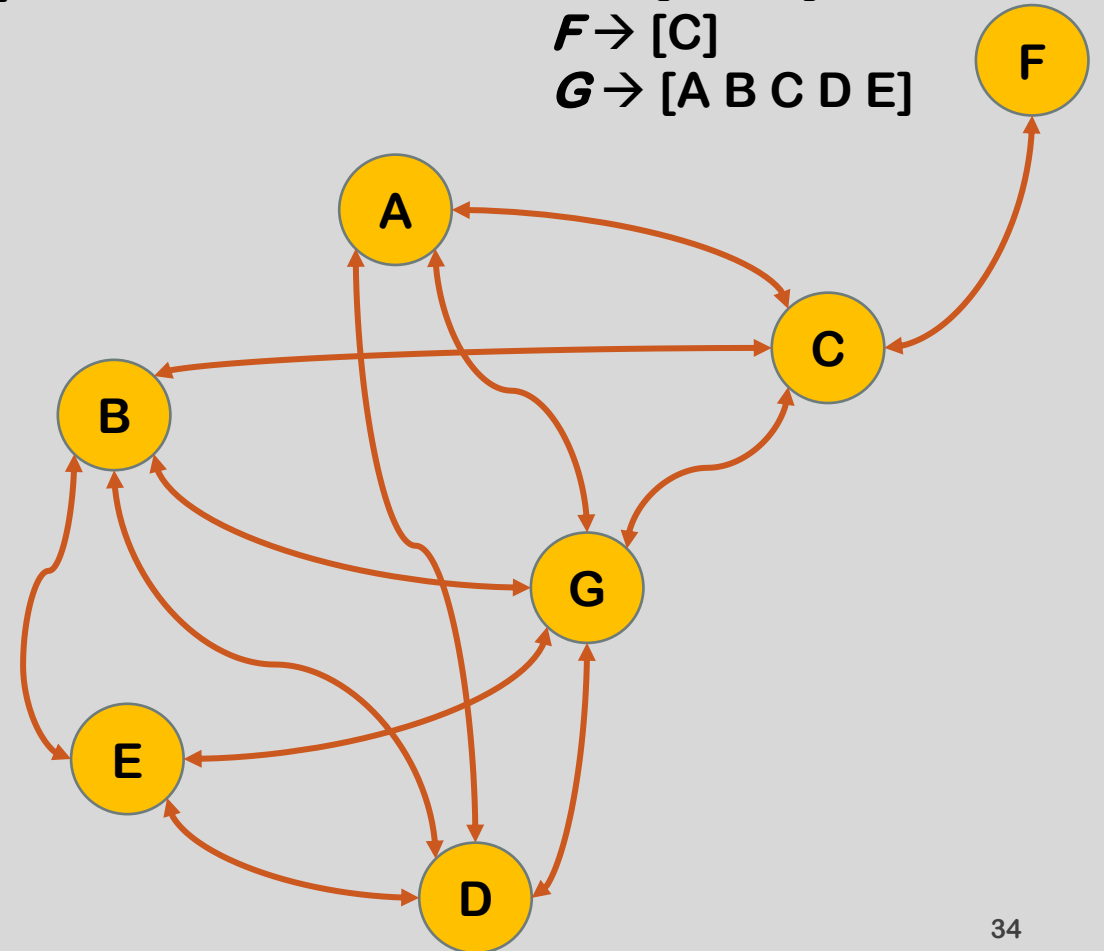
$(D F) \rightarrow []$

$(D G) \rightarrow [A B E]$

$(E F) \rightarrow []$

$(E G) \rightarrow [B D]$

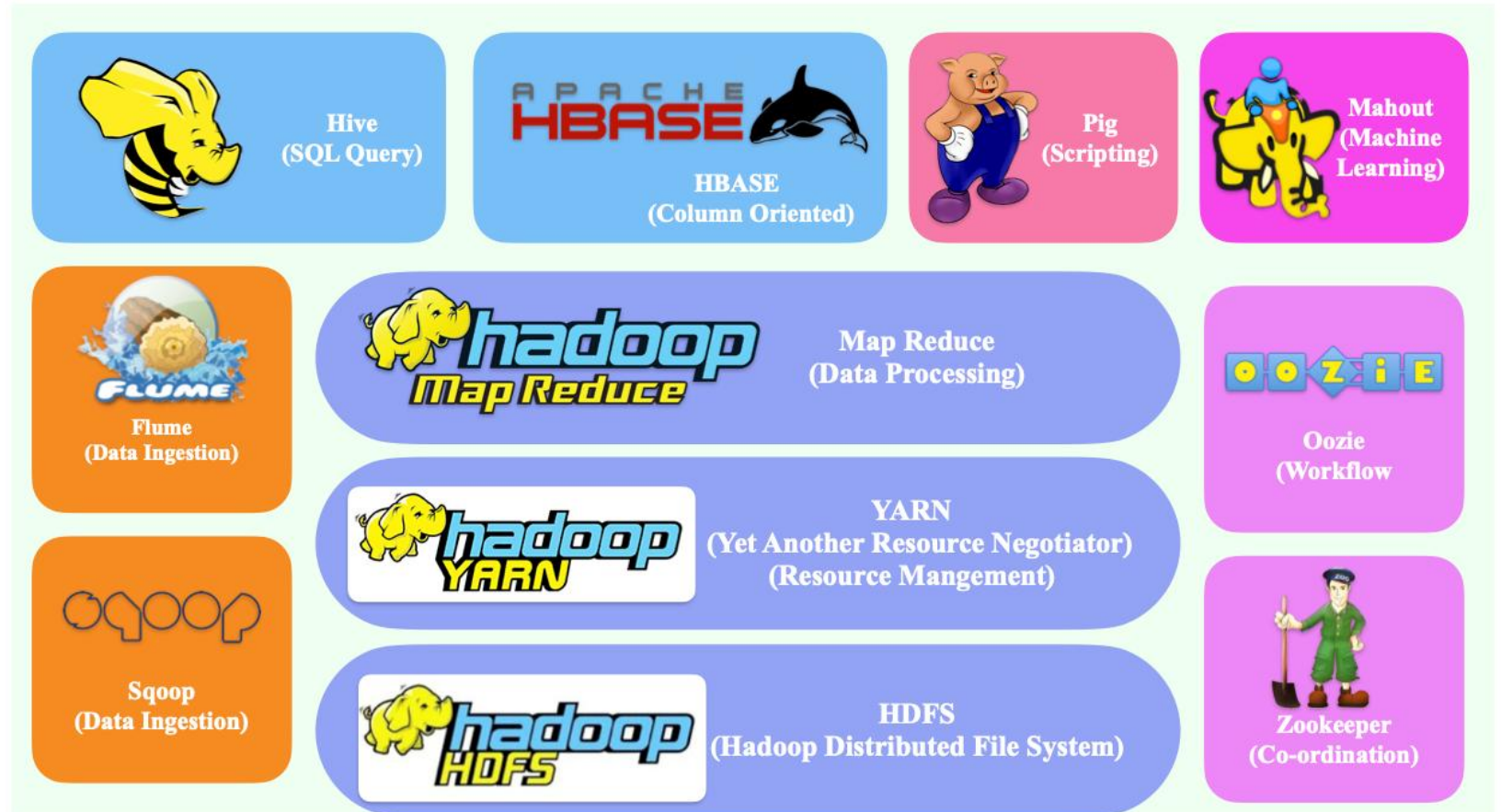
$(F G) \rightarrow [C]$



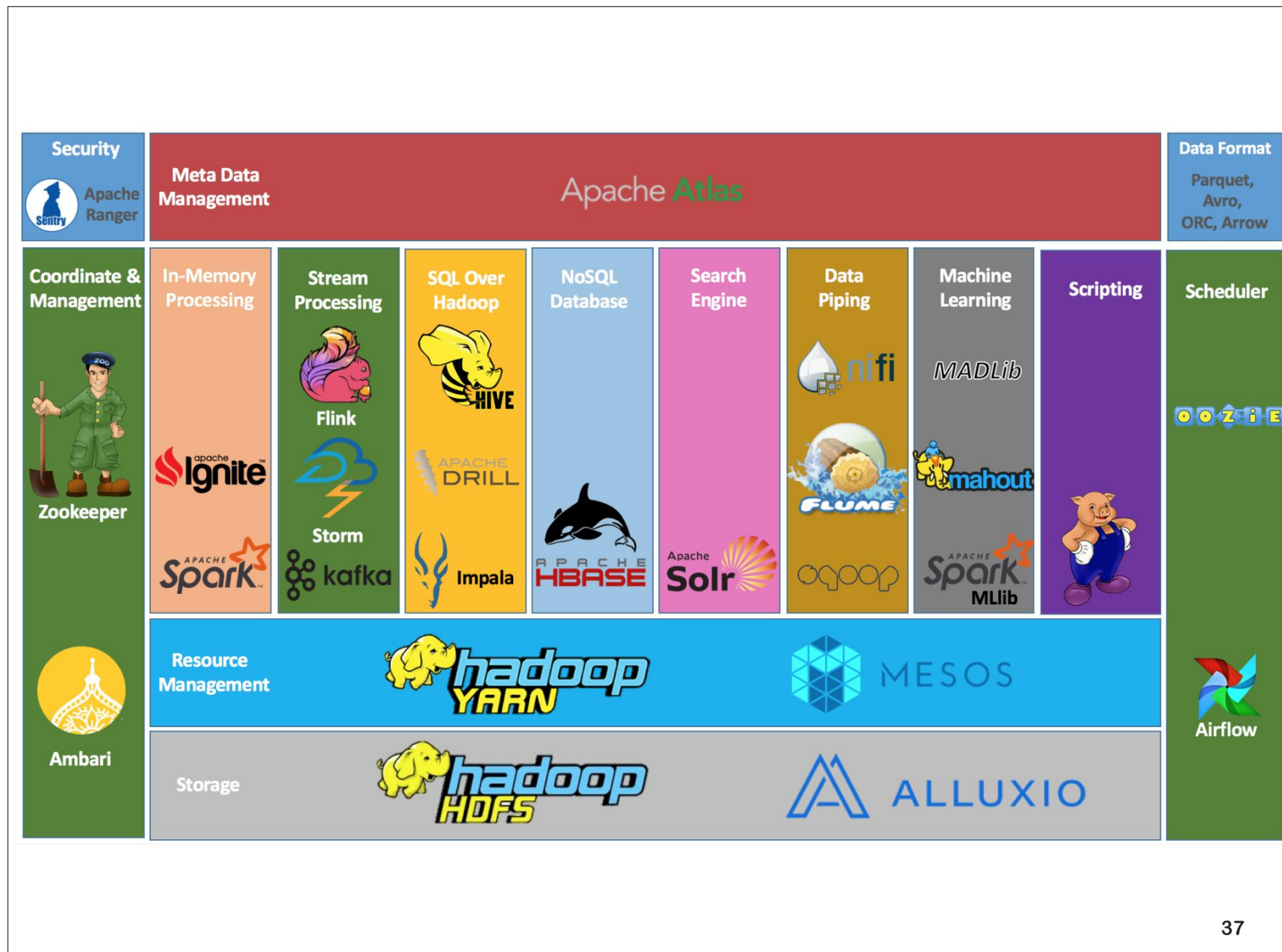


HADOOP ECOSYSTEM

Hadoop Ecosystem



Hadoop Ecosystem



How Does the Ecosystem Projects Built on Hadoop?

Built on only HDFS.

Built on HDFS and MapReduce.

Designed to integrate with or support Hadoop.

Data Storage: HBase



- Hadoop Database
- NoSQL datastore
- Capable of storing massive data (Peta bytes+)
- High write throughput (Scales to 100,000+ if inserts per second.)

Data Storage: HBase



- **Handles sparse data well**
 - **No wasted spaces for empty columns in a row.**
- **Limited access model**
 - **Optimised for lookup of a row by key rather than full queries.**
 - **Single row operations only. (No transactions)**
 - **Only one column can be indexed. (the “row key”)**

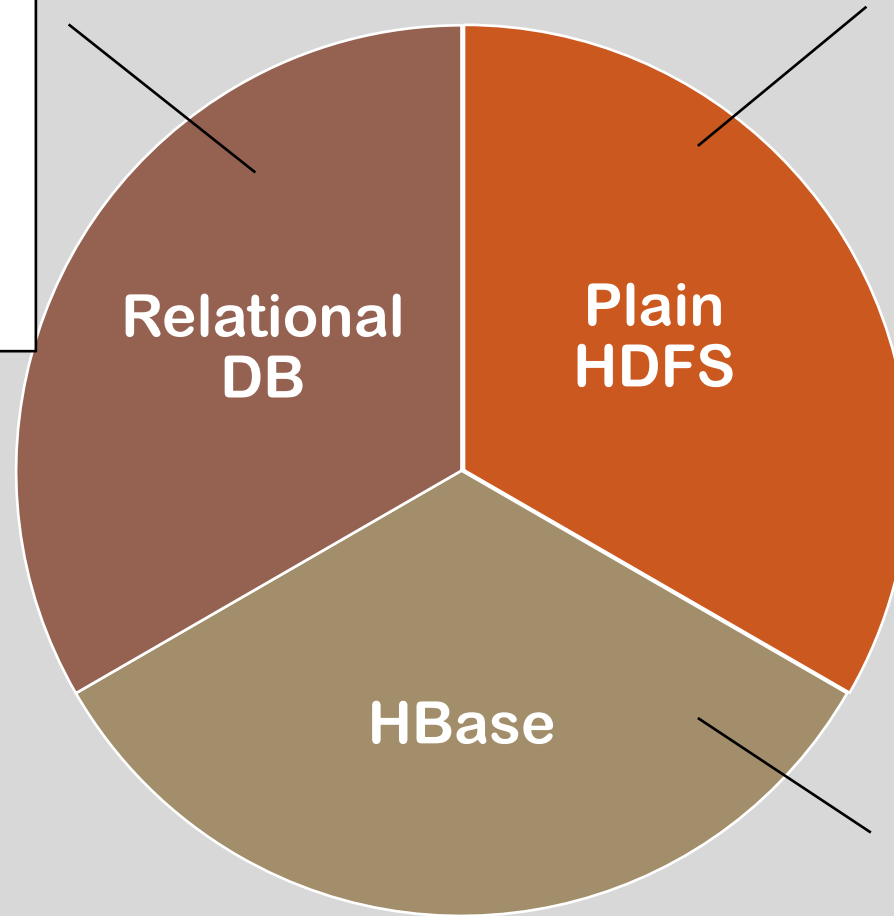
HBase vs Relational Database

	Relational Database	Apache HBase
Data Layout	Row-oriented	Column-oriented
Transactions	Yes	Single row only
Query Language	SQL	get/put/scan or using Hive or Impala
Security	Authentication and Authorisation	Kerberos Protocol
Indexing Method	Any column	Row-key only
Maximum Data Size	TBs	PB+
Read/Write throughput	Thousands	Milions



How to Choose Your System?

- Full transaction is needed.
- Real-time query capability is needed.



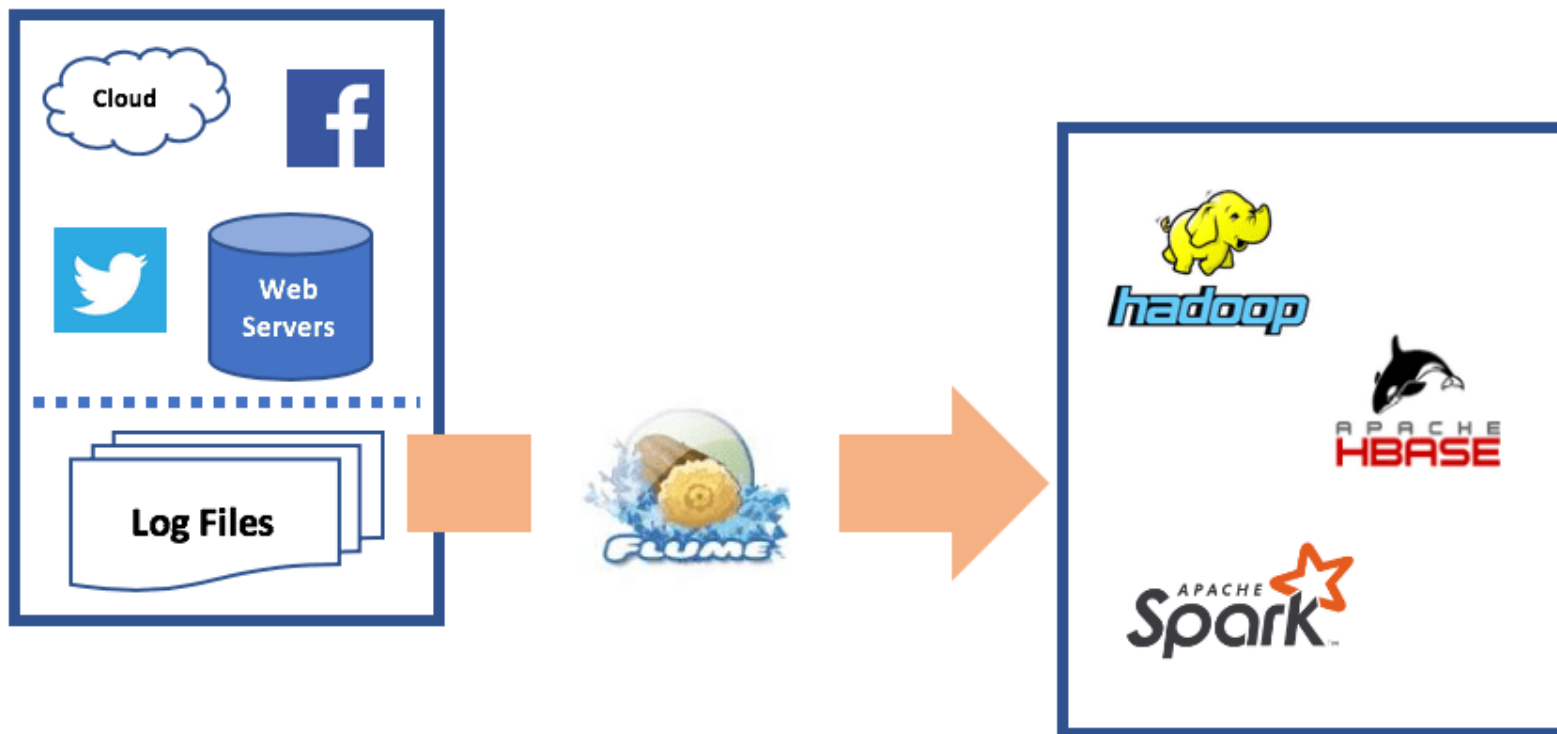
- Appending your dataset. (no random write)
- Usually read the whole dataset. (no random read)

- Random read/write is required.
- Having thousands of operations per second on TB+ of data.



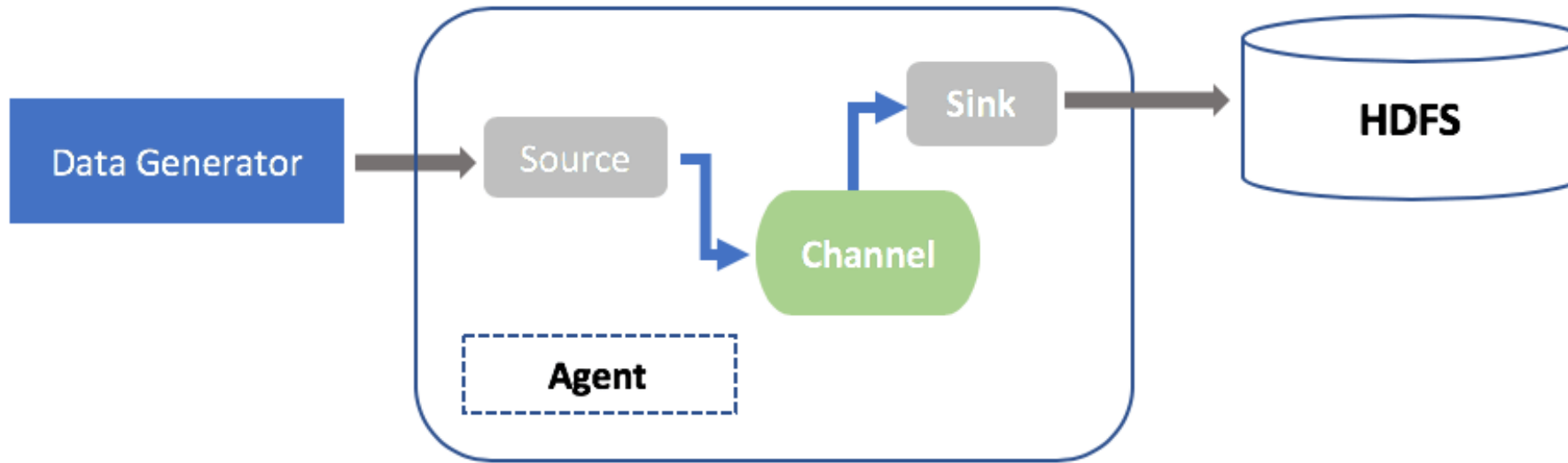
Data Integration: Flume

- Apache Flume is a service to move large amounts of data in real-time.
- It imports data into HDFS as it is generated after batch-processing.
- Characteristics:
 - Distributed
 - Reliable and available
 - Horizontally Scalable
 - Extensible



Flume's Feature and Capacity

- Apache Flume pulling logs from multiple sources and shipping them to Hadoop, Apache HBase and Apache Spark.



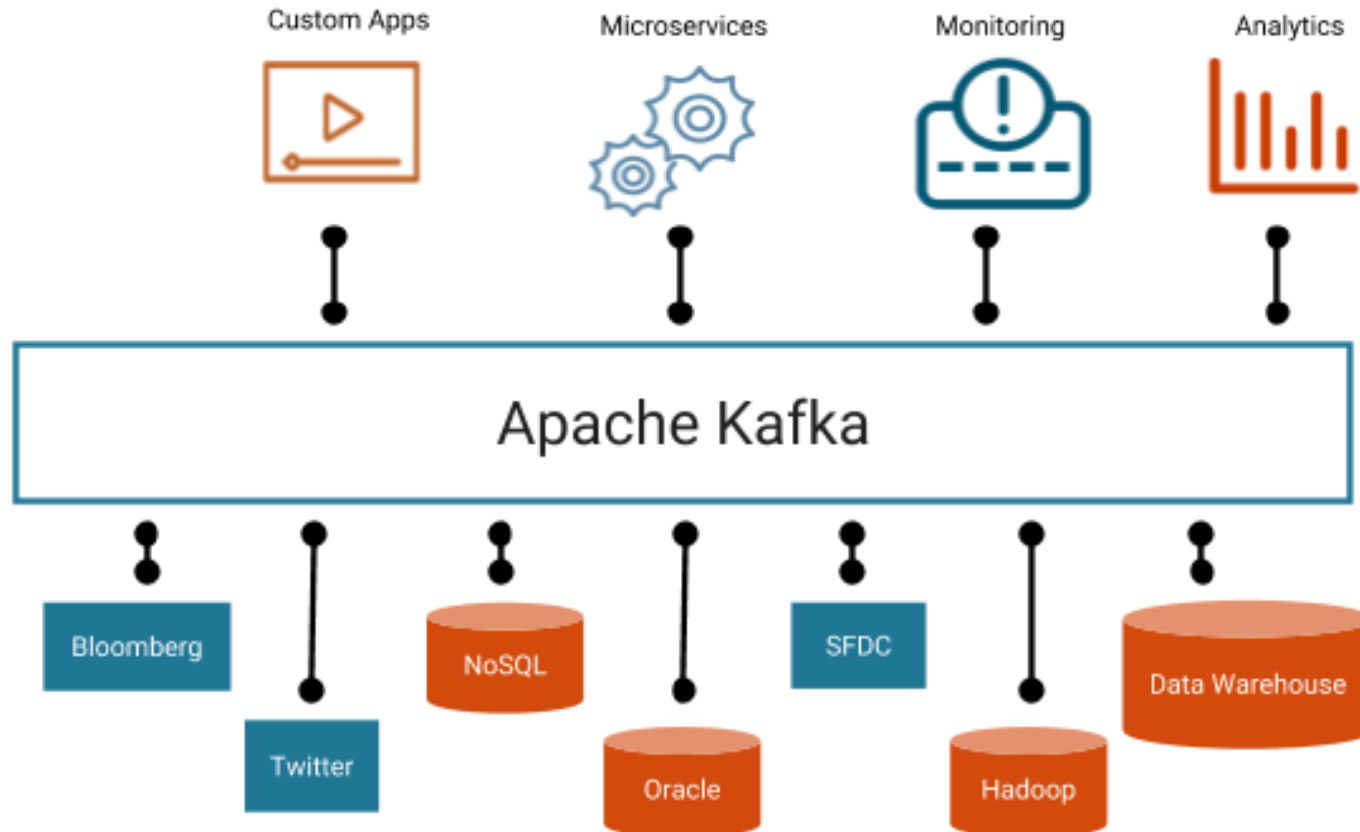
Flume Architecture

- Important parts of flume:
 - Data generating sources.
 - The Flume agent.
 - The destination or target.



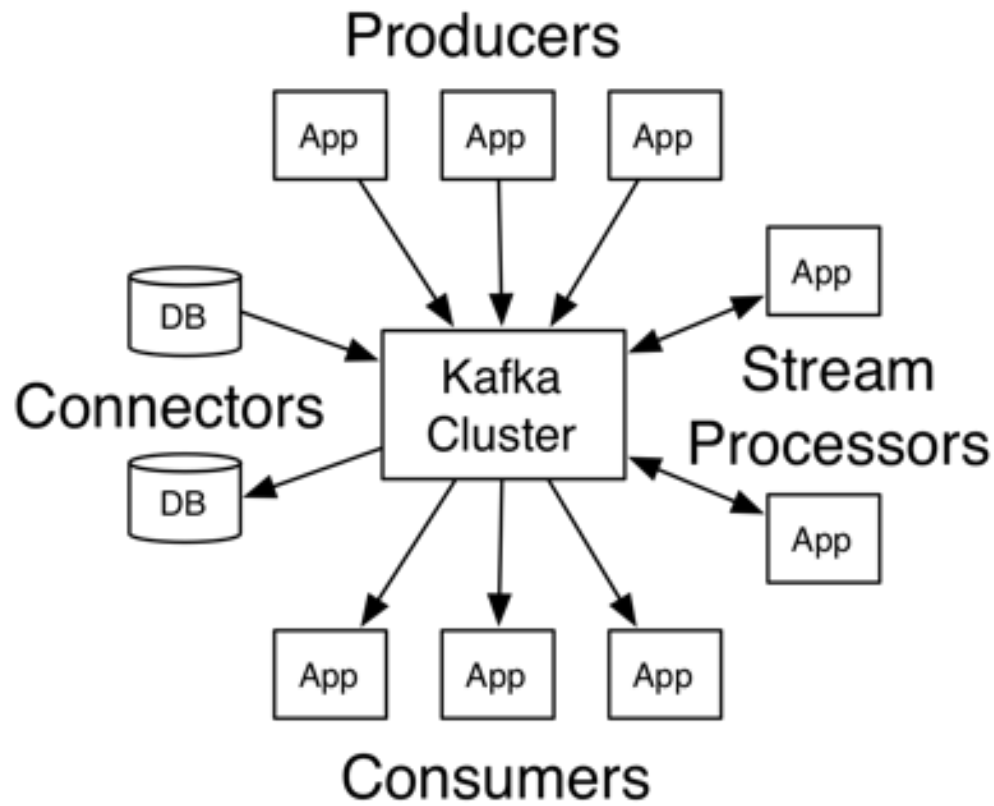
Data Integration: Kafka

- An open source platform developed by *LinkedIn* to handle real-time data.
- Kafka publishes and subscribes a stream of records.
- Kafka stores, reads and analyses the streaming data where developers and users contribute the coding updates.



Kafka's Feature and Capacity

- Kafka's simple structure ensures its reliability, durability, and performance.



- Some of the use cases where Kafka is widely used are:
 - Track activities on a website.
 - Stream processing.
 - Collecting and monitoring metrics.
 - Log Aggregation.

KAFKA ARCHITECTURE

Flume vs Kafka

	Flume	Kafka
Meanings	• A tool to collect log data from distributed web servers. • Collected data will land into HDFS/HBase/SPARK for further analysis. • Highly reliable and configurable. • Highly efficient and robust in processing log files, both in batch and real-time processing.	• Runs as a cluster and handles incoming high volume data streams in real-time. • Composed of three components: the publisher, Kafka cluster/manager, and subscriber. • Stores stream of records into different categories or topics. • Each record in Kafka will be stored as a log entry where the receiver/sender will not be aware of each other.

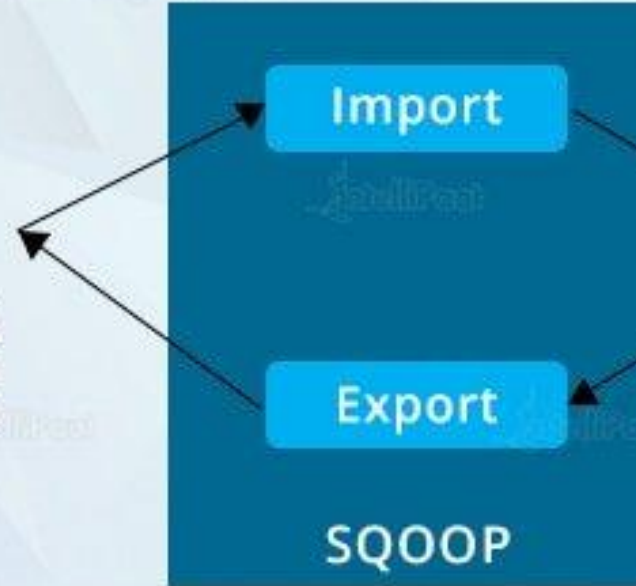
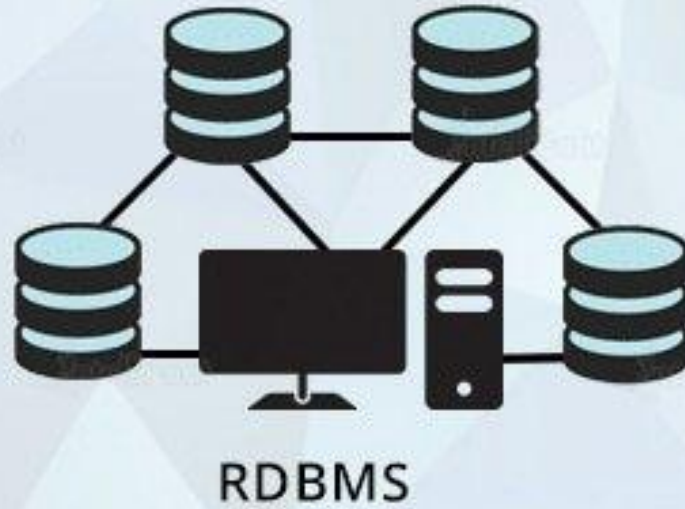
Flume vs Kafka

	Flume	Kafka
Concepts	• Capable of taking in streaming data from multiple sources for storage and analysis for use in HBase or Hadoop. • Ensures guaranteed data delivery because both the receiver and sender agents evoke the transaction to ensure guaranteed semantics. • It can scale horizontally.	• Treating each topic partition as an ordered set of messages. • Based on publish-subscribe architecture and does not track the identity of subscriber/publisher. • Retaining all messages or data as logs where subscribers are responsible to track the location in each log. • Capable of supporting large number of publishers and subscribers and store large amounts of data.

Flume vs Kafka

	Flume	Kafka
Basic of Formation	• Flume is a service or tool for gathering data into Hadoop/HBase/SPARK.	• Kafka is an efficient, fault-tolerant and scalable messaging system.
Application Areas	• Process transaction logs in application servers, web servers, etc. For example, e-commerce, online retail portals, social media, etc.	• Monitor data from distributed applications. • Make data available to multiple subscribers based on their interests. • Log aggregation service.
Approach	• Need to gather big data either in streaming or in batch mode from different sources. • Efficient when working with logs.	• Kafka is required to efficiently process real-time data streams without data loss. • Need to ensure data delivery even during machine failures, hence it is a fault-tolerant system.

Facilitating Data Transfer Between RDBMS and Hadoop



Bridging RDBMS with HDFS: Sqoop

- Sqoop transfers data between RDBMS and HDFS.
- Supports JDBC, ODBC, and several specific databases.
- “Sqoop” = “SQL to Hadoop”



Sqoop Custom Connectors

- **Custom connectors for:**
 - **MySQL**
 - **Postgres**
 - **Netezza**
 - **Teradata**
 - **Oracle (partnered with Quest Software)**
- **Not open source, but free to use.**



Data Processing: Apache Spark

- Spark is a fast, general engine for large-scale data processing on a cluster.
- Originally developed at UC Berkeley's AMPLab.
- Open source Apache project.
- Provides several benefits over MapReduce.



Data Processing: Apache Spark

- **Benefits over MapReduce:**
 - **Faster**
 - **Better suited for iterative algorithms**
 - **Can hold intermediate data in RAM, resulting in much better performance.**
 - **Easier API**
 - **Supports Python, Scala, and Java.**
 - **Supports real-time streaming data processing**

Spark vs MapReduce

Spark	MapReduce
• Flexible • Elegant • Fast • Supports real-time streaming data processing • Can hold intermediate data in RAM.	• Widely used, huge investment already made • Supports and supported by many complementary tools. • Mature, well-tested.

- Spark is expected to supplant MapReduce as the general processing framework used by most organisations.



Apache Pig

Data Analysis: Hive and Pig

- MapReduce is powerful but hard to master.
- Solutions: Hive and Pig.
 - Languages for querying and manipulating data.
 - Leverage existing skillsets.
 - Open source Apache projects.
(Hive is initially developed at Facebook.)
(Pig is initially developed at Yahoo!)
- Interpreter runs on a client machine.



Hive

- HiveQL is an SQL-like interface to Hadoop.
- Example:
**SELECT * FROM users
WHERE region_code > 2
ORDER BY UID**



Apache Pig

Apache Pig

- Apache Pig is a dataflow language for transforming large datasets.

- Example:

Purchases = LOAD

**“/user/ret01/purchases” AS (itemID,
price, storeID, purchaserID);**

**Bigticket = FILTER purchases BY
price > 1000;**

Hive vs Pig

	Hive	Pig
Language	HiveQL (SQL-like)	Apache Pig (dataflow language)
Schema	Table definitions stored in a metastore.	Schema optionally defined at runtime.
Programmatic Access	JDBC, ODBC	PigServer (Java API)

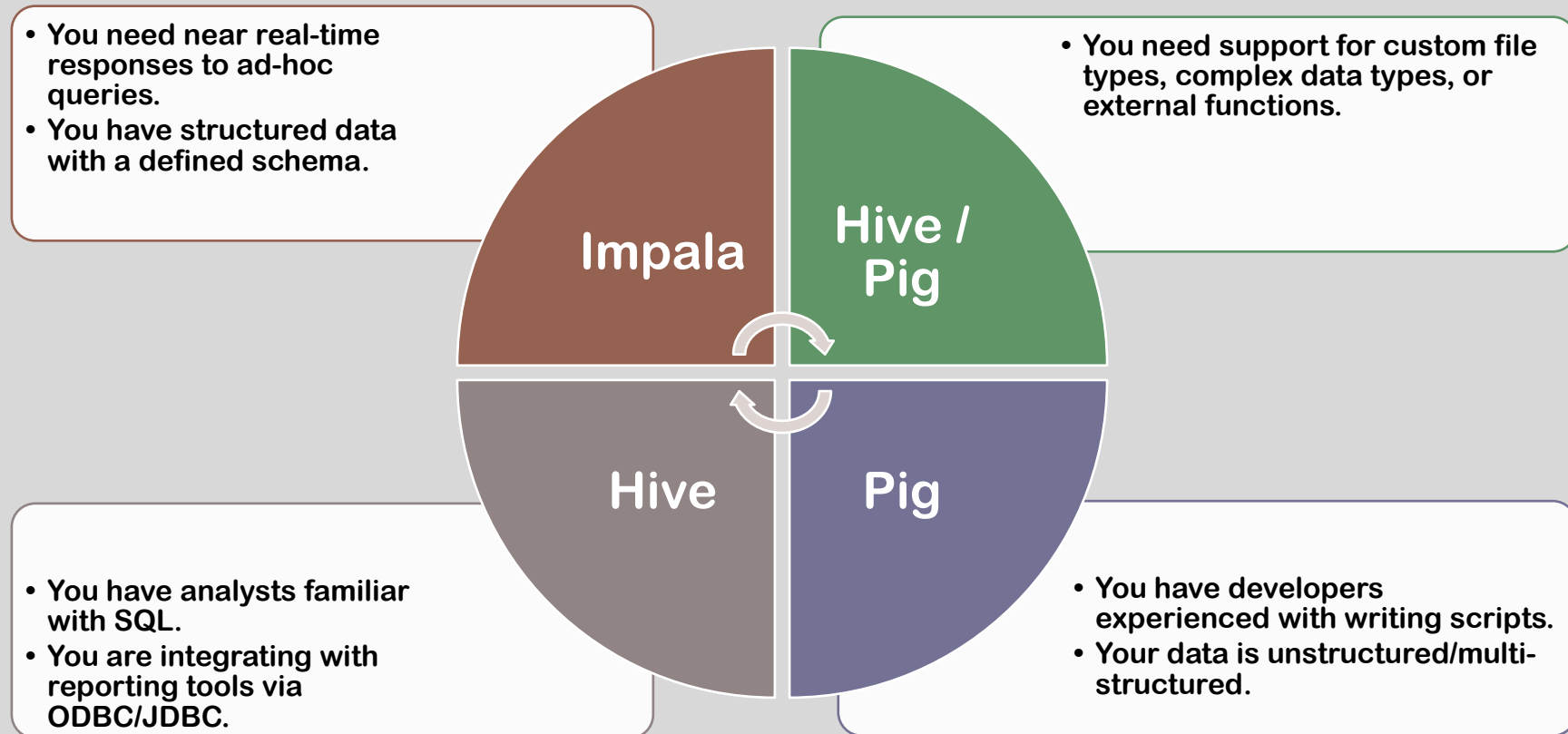
- JDBC: Java Database Connectivity
- ODBC: Open Database Connectivity

High Performance Queries - Impala

- **High-performance SQL engine for vast amounts of data**
 - Similar query language to HiveQL.
 - 10 to 50+ times faster than Hive, Pig, or MapReduce
- **Impala runs on Hadoop clusters**
 - Data stored in HDFS
 - Does not use MapReduce
- **Developed by Cloudera**
 - 100% open source, released under the Apache software license

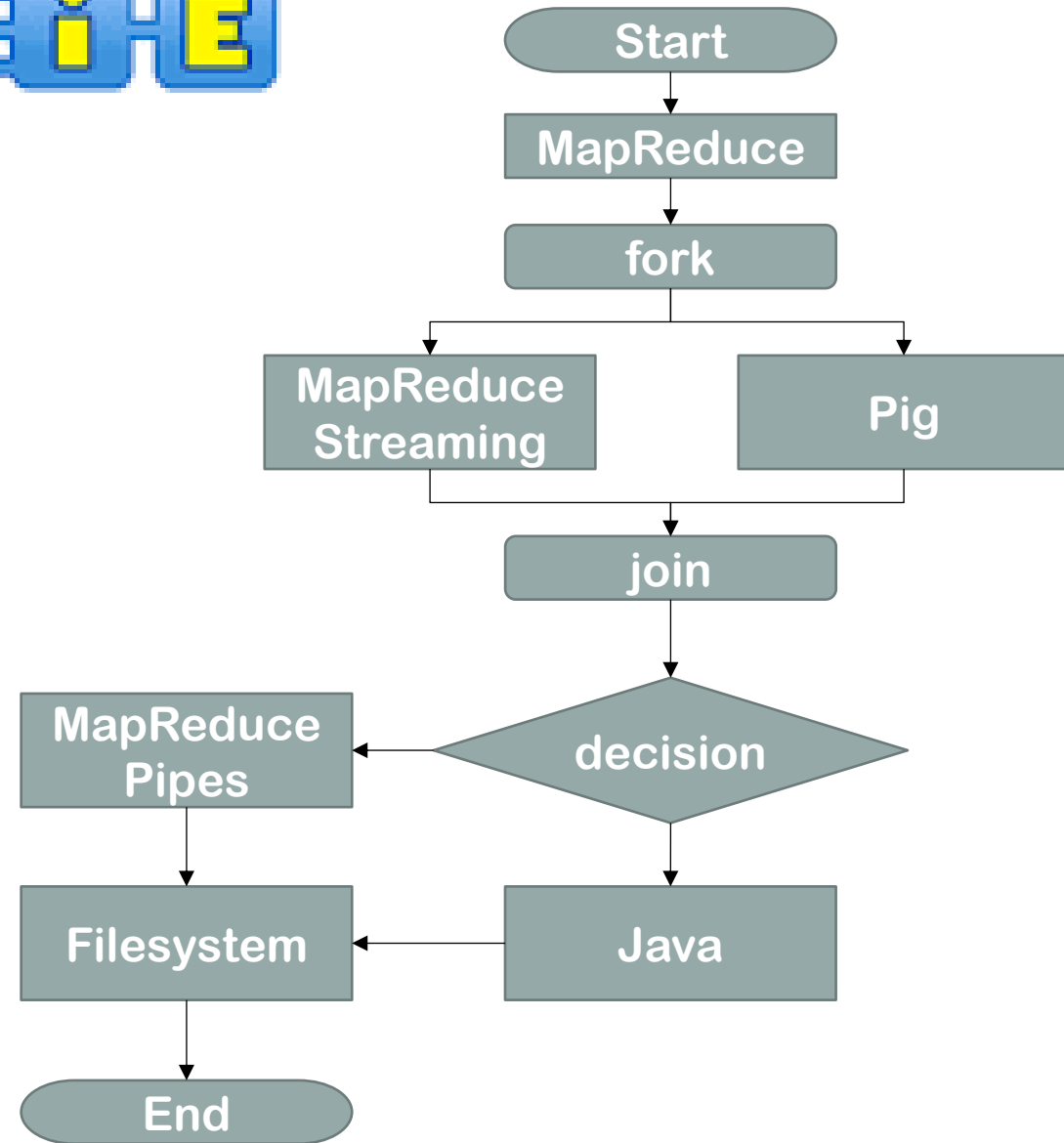


Recommendation of Usage



Oozie – A Workflow Engine

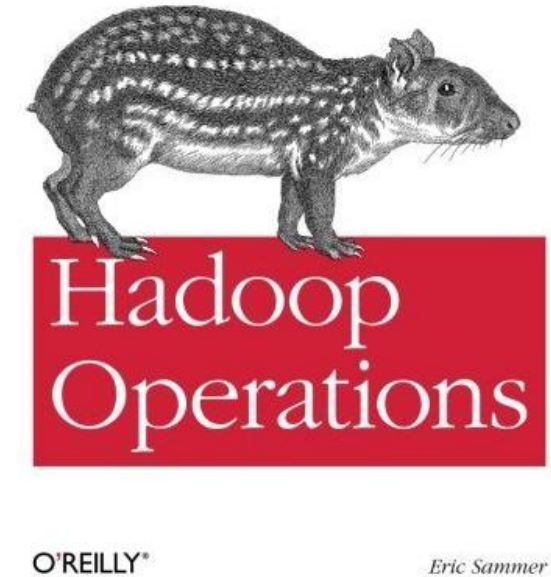
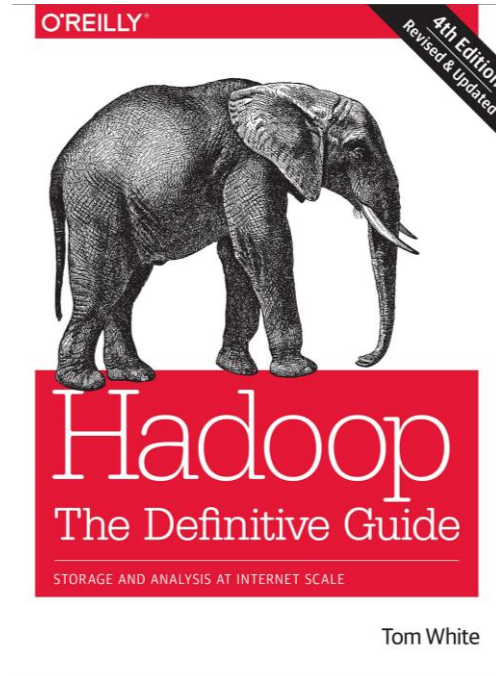
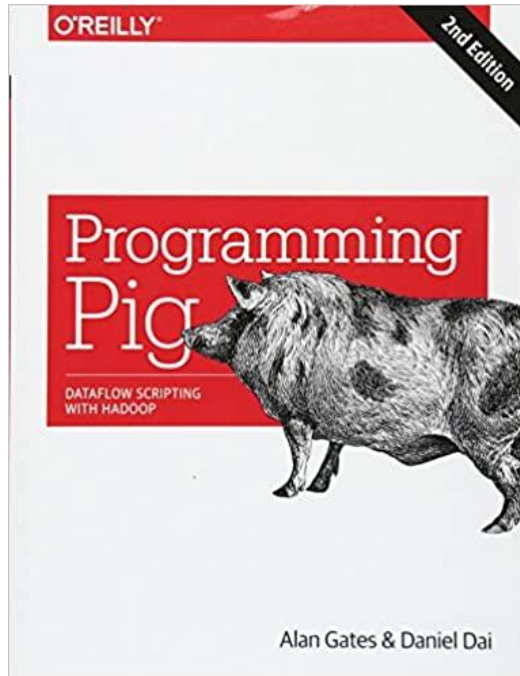
- Oozie is a workflow engine for MapReduce jobs.
- It defines the dependencies between jobs.
- The Oozie server submits the jobs to the server in the correct sequence.



Mahout – The Machine Learning Library

- Mahout is a Machine Learning (ML) library written in Java.
- It can be used for:
 - Collaborative filtering. (Recommendations)
 - Clustering (Finding naturally occurring “groupings” in data).
 - Classification (determining whether new data fits a category)





Texts and Resources

- Unless stated otherwise, the materials presented in this lecture are taken from:
 - Hadoop: the definitive guide, White, Tom (Tom E.) author., 4th ed., Sebastopol, California: O'Reilly, 2015.
 - Hadoop operations, Sammer, Eric., Loukides, Michael Kosta.; Nash, Courtney.; Romano, Robert (Illustrator), illustrator., First edition., Sebastopol, CA: O'Reilly, 2012.
 - Programming pig: dataflow scripting with hadoop, Gates, Alan, author.; Dai, Daniel, 2nd edition., Beijing, China: O'Reilly, 2017



Unit Summary

- Unit structure
- Requirements, Expectations, and Rules
- HDFS

Summary

